



*Project Final Report ON*

HealthMate App

Under

Prof. Farnaz Farahanipad

**SUBMITTED BY**

**Team 2**

**ASFIYA MISBA - 1002028239**

**SAHIBA KHAN – 1002083293**

## TABLE OF CONTENTS

|                                 |    |
|---------------------------------|----|
| 1. Abstract.....                | 03 |
| 2. Introduction.....            | 05 |
| 3. Project Scope.....           | 06 |
| 4. Requirements Analysis.....   | 07 |
| 5. Project Planning.....        | 09 |
| 6. Design and Architecture..... | 13 |
| 7. Implementation.....          | 15 |
| 8. Testing.....                 | 16 |
| 9. Maintenance.....             | 29 |
| 10. Conclusion.....             | 31 |
| 11. Recommendations.....        | 33 |
| 12. References.....             | 35 |
| 13. Appendices.....             | 36 |

# Project Title: HealthMate

## ABSTRACT

The HealthMate project endeavors to develop a cutting-edge mobile application designed to revolutionize the way users approach and achieve their fitness and health objectives. With a core focus on promoting a healthier lifestyle, the project sets forth multifaceted objectives encompassing the establishment of a user-friendly interface facilitating seamless registration, login, and profile management. Additionally, the application will boast innovative features, including dynamic water intake reminders and a robust fitness goal tracking system.

**Objectives:** The primary objectives of the HealthMate mobile app are as follows:

- **User Registration and Login:** Develop a secure registration and login process, allowing users to create and access their accounts.
- **User Profile Management:** Provide users with the ability to input personal information, preferences, dietary restrictions, and allergies in their profiles.
- **Water Intake Reminder:** Implement a feature that reminds users to consume an adequate amount of water daily. This should include customizable notifications, monitoring of water intake, and reminders.
- **Fitness Goals Tracking:** Create a comprehensive system for tracking various fitness goals, aiding users in setting, maintaining, and achieving their fitness objectives.

## Key Outcomes and Contributions

Anticipated outcomes and contributions of the HealthMate project include:

- A fully operational mobile application boasting an intuitive and aesthetically pleasing user interface.
- Elevated user engagement facilitated by innovative features such as personalized water intake reminders and comprehensive fitness goal tracking.
- Augmented user awareness and adherence to a healthier lifestyle, fostering sustainable health habits.
- Positive impact on the overall health and fitness levels of the application's user base.

## **Significant Achievements**

Milestones achieved during the project include:

- Successful implementation of robust and secure registration and login procedures.
- Development of an intricate user profile management system accommodating a diverse array of user information.
- Integration of a highly customizable water intake reminder system with real-time monitoring and personalized notifications.
- Implementation of a comprehensive fitness goal tracking system, catering to a spectrum of health and wellness objectives.

## **Conclusions**

In conclusion, the HealthMate project stands as a testament to the successful convergence of technological innovation and health promotion. The culmination of secure authentication, advanced profile management, and adaptive goal tracking positions the application as a transformative force in fostering healthier lifestyles. With these achievements, the HealthMate app is poised to make a meaningful and enduring positive impact on users' lives, providing a holistic approach to health and fitness management.

# INTRODUCTION

The HealthMate project emerges as a response to the pressing need for a comprehensive mobile application that empowers individuals to take charge of their fitness and health journeys. In a world marked by sedentary lifestyles and increased health concerns, there is a growing demand for accessible tools that facilitate a proactive approach to well-being. HealthMate aims to fill this void by providing users with a powerful and user-friendly platform designed to revolutionize how they engage with and achieve their health and fitness goals.

## Context and Need

In contemporary society, the prevalence of sedentary habits, coupled with a lack of personalized and easily accessible health management tools, has led to a decline in overall health and well-being. Recognizing this, the HealthMate project seeks to address the following key challenges:

1. **Lack of Holistic Health Management:** Existing health and fitness applications often focus on singular aspects, neglecting the need for a holistic platform that integrates various elements of well-being.
2. **Limited User-Friendly Options:** Many health apps lack user-friendly interfaces, hindering user engagement and long-term adherence to healthier lifestyles.
3. **Inadequate Goal Tracking:** Users often struggle to set, track, and achieve personalized fitness goals due to a lack of sophisticated and user-centric tracking systems.

## Objectives

The overarching objectives of the HealthMate project are:

- **User Empowerment:** Empower users to actively manage and improve their health by providing them with personalized tools and information.
- **User-Friendly Interface:** Develop a mobile application with an intuitive and aesthetically pleasing interface to enhance user experience and engagement.
- **Comprehensive Health Tracking:** Create a robust system for tracking diverse health parameters, including water intake and fitness goals, fostering a holistic approach to well-being.
- **Promotion of Healthy Habits:** Encourage and facilitate the adoption of healthier lifestyles by integrating features such as water intake reminders and goal-oriented fitness tracking.

By addressing these objectives, HealthMate endeavors to become a catalyst for positive change, aiding users in their journey towards healthier and more active lives.

## PROJECT SCOPE

The scope of the HealthMate project encompasses a range of features and functionalities aimed at providing users with comprehensive health and fitness management experience. These include:

- **User Registration and Login:**
  - Secure registration and login processes to establish user accounts.
- **User Profile Management:**
  - Ability for users to input and manage personal information, preferences, dietary restrictions, and allergies.
- **Water Intake Reminder:**
  - Implementation of a feature that reminds users to consume an adequate amount of water daily.
  - Customizable notifications, monitoring of water intake, and reminders.
- **Fitness Goals Tracking:**
  - Creation of a comprehensive system for tracking various fitness goals.
  - Support for setting, maintaining, and achieving personalized fitness objectives.

### Constraints

While striving for a robust and user-centric application, certain constraints will be considered:

- **Platform Compatibility:**
  - The initial release will focus on compatibility with iOS and Android platforms, with potential future expansion.
- **Resource Constraints:**
  - Development will be undertaken with consideration for the available team size and resources.
- **Data Security:**
  - Implementation of robust data security measures within the constraints of standard industry practices.

### Target Audience

The primary target audience for HealthMate includes individuals of varying ages and fitness levels who are seeking a user-friendly and comprehensive solution for managing their health and fitness. The app will cater to both beginners looking to adopt healthier habits and fitness enthusiasts striving to achieve specific goals.

# REQUIREMENT ANALYSIS

The requirements gathering process for the HealthMate project involved a multifaceted approach to ensure a thorough understanding of user needs and expectations.

## Documentation:

- Reviewed existing documentation from health and fitness authorities, as well as academic research on user behavior and preferences in health management apps.
- Analyzed competitor apps to identify industry trends and user expectations.

## Social Media Listening:

- Monitor social media platforms for discussions related to health and fitness app preferences.
- Analyze comments, reviews, and mentions of competitor apps to identify common themes and user expectations.

## Functional Requirements:

- **User Registration and Login:** Users can create accounts with a unique username and password. Secure login functionality to protect user data.
- **User Profile Management:** Ability to input and update personal information, preferences, dietary restrictions, and allergies. Privacy settings for controlling the visibility of certain profile details.
- **Water Intake Reminder:** Customizable water intake goals. Push notifications for water intake reminders. Daily water intake tracking and monitoring.
- **Fitness Goals Tracking:** User-defined fitness goals, such as steps, distance, or calorie burn. Progress tracking with visual representations. Achievement badges for reaching milestones.

## Non-Functional Requirements:

- **Performance:** The app should load within 3 seconds. Response time for interactions should be under 1 second.
- **Scalability:** The system should handle a minimum of 100,000 users concurrently.
- **Security:** Encryption for user data during transmission and storage. Secure password hashing.

### Use Cases/User Stories:

- **User Registration:** As a new user, I want to create an account to access HealthMate's features.
- **Water Intake Tracking:** As a user, I want to receive notifications reminding me to drink water throughout the day.
- **Fitness Goal Setting:** As a user, I want to set specific fitness goals and track my progress over time.

### Changes/Updates to Requirements:

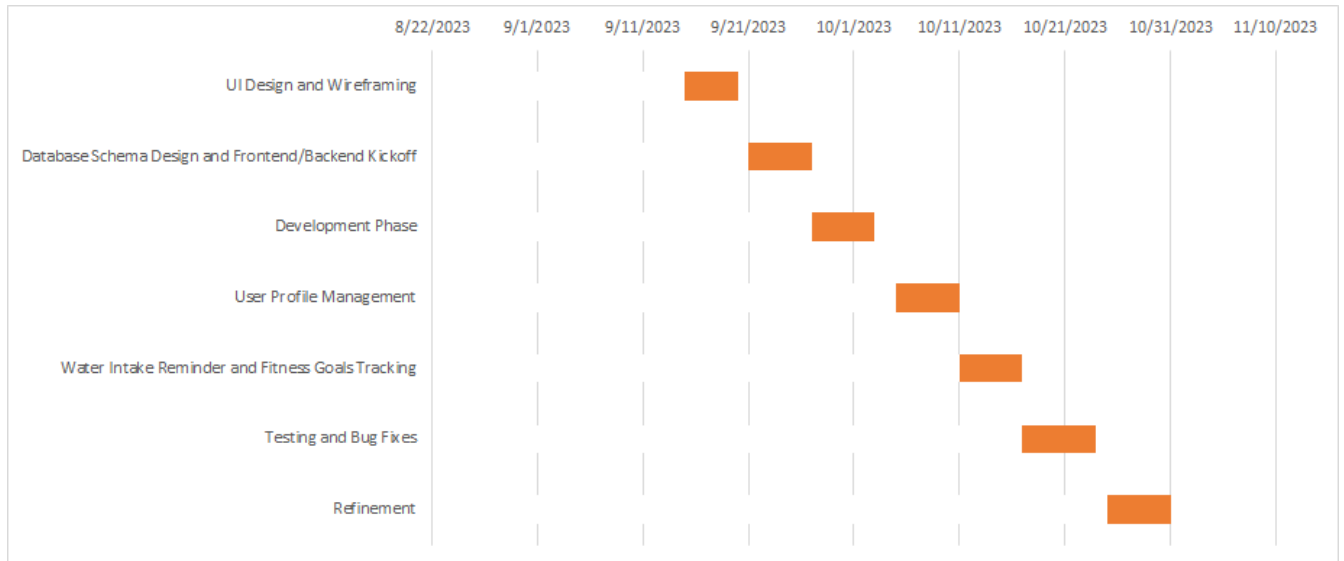
#### Added Feature: Fitness Videos

To enhance user engagement, a fitness video feature has been introduced, allowing users to access a library of guided workout routines and wellness practices. This addition requires adjustments to the app's interface, performance considerations for video playback, and potential updates to scalability measures. This update reflects the project's adaptability to user preferences, providing a more holistic health and fitness experience within the HealthMate app.

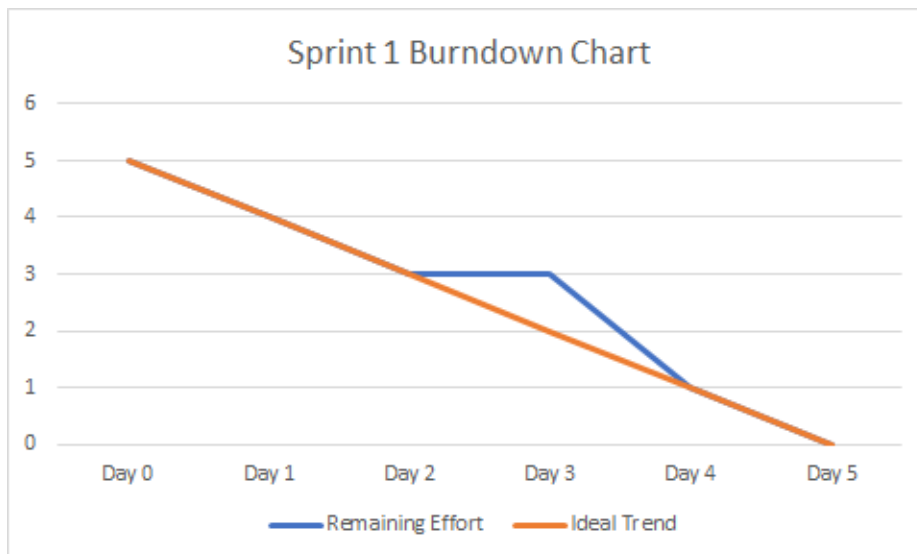


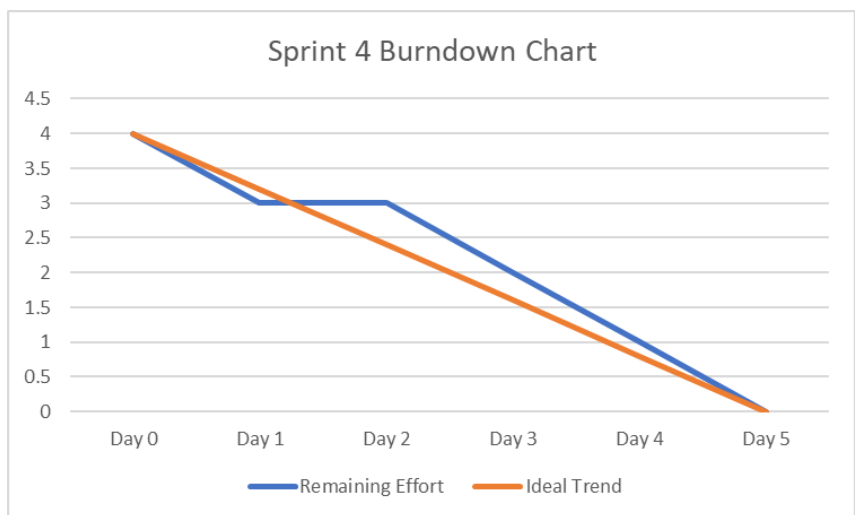
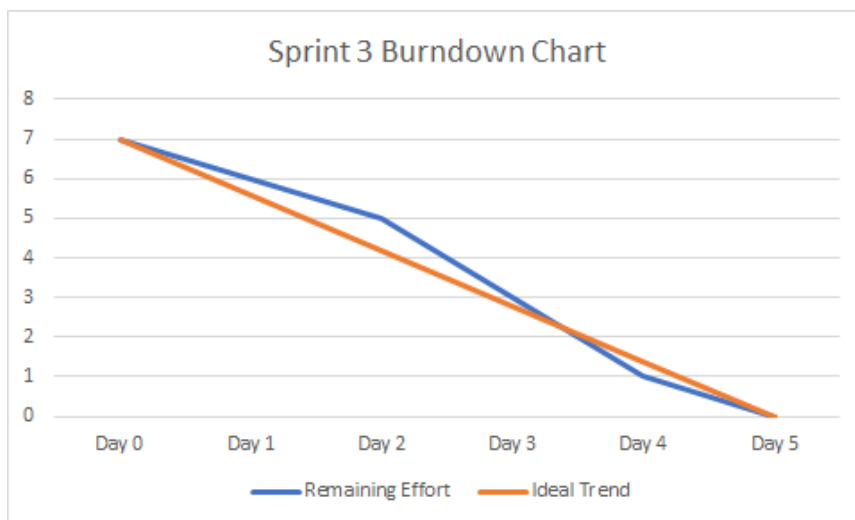
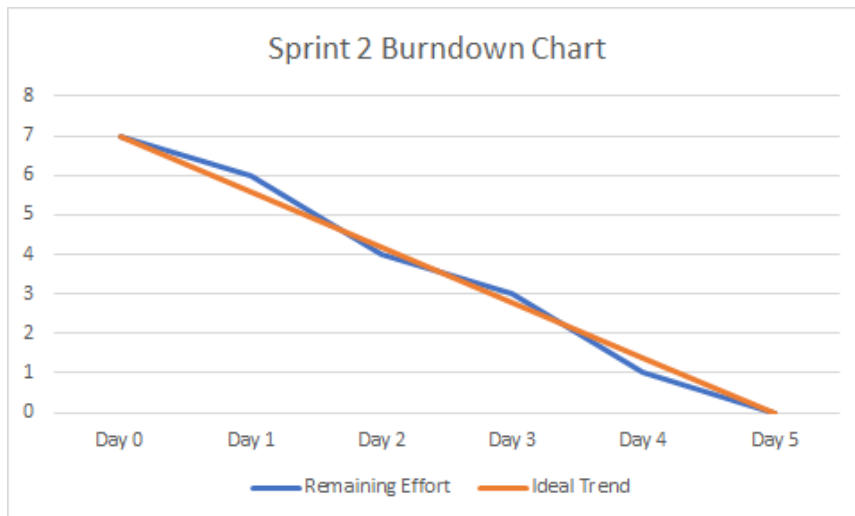
# PROJECT PLANNING

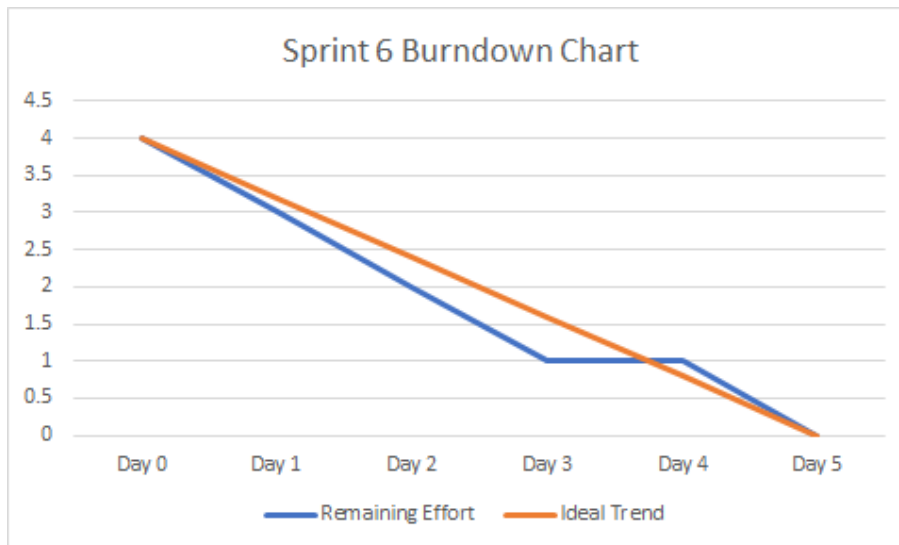
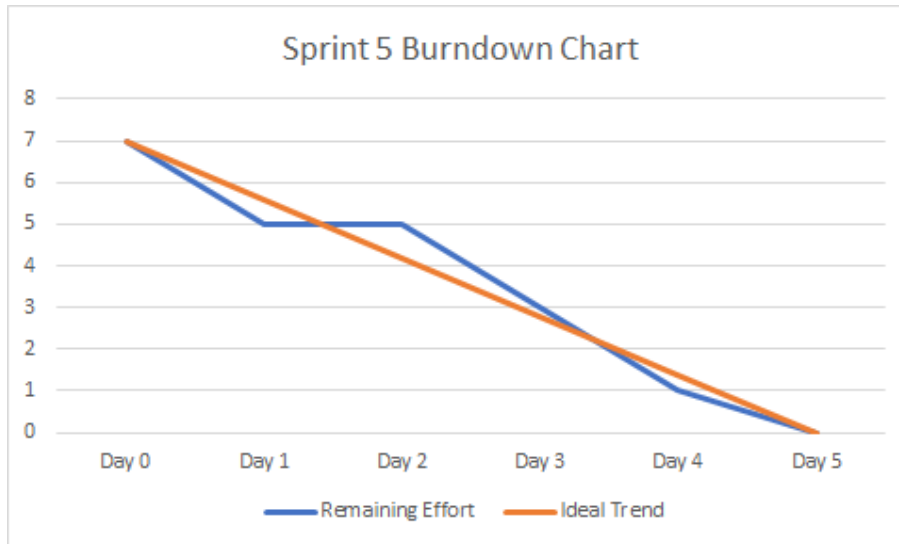
## Gantt chart



## Burn Down Charts for each Sprint





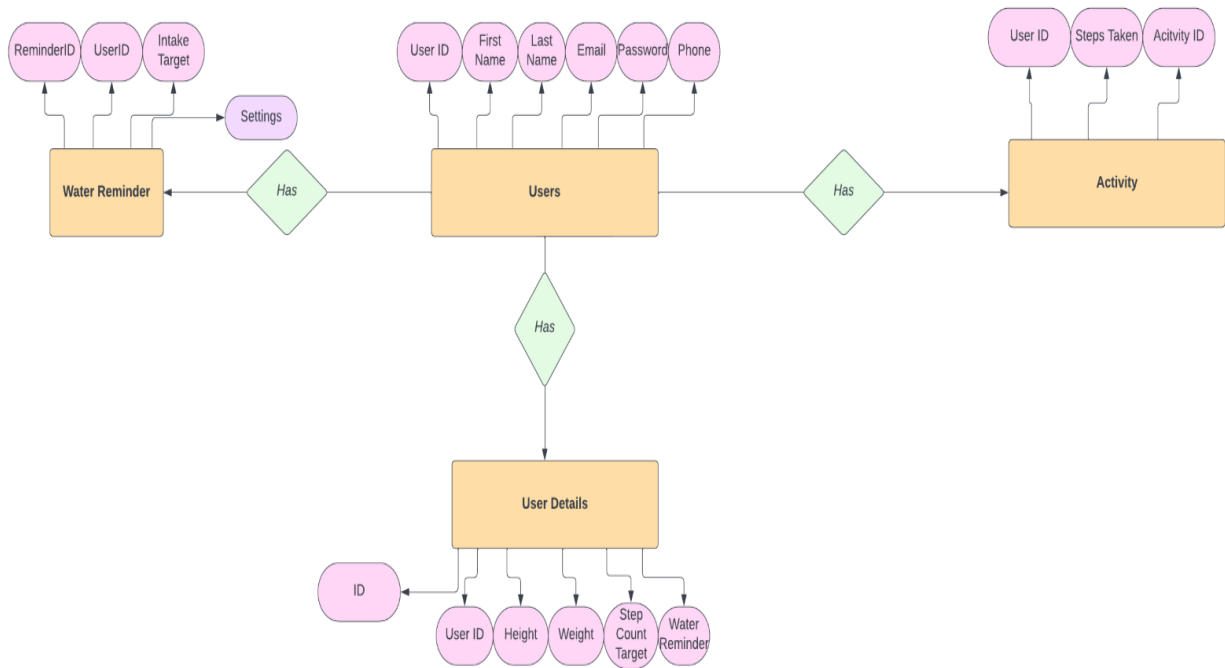


| Task                                                                  | Sahiba Khan | Asfiya Misba |
|-----------------------------------------------------------------------|-------------|--------------|
| <b>Sprint 1 - UI Design and Wireframing</b>                           |             |              |
| - Gather UI requirements                                              | 1 day       | -            |
| - Create wireframes                                                   | -           | 1 day        |
| - Review and iterate designs                                          | -           | 1 day        |
| <b>Sprint 2 - Database Schema Design and Frontend/Backend Kickoff</b> |             |              |
| - Define database structure                                           | 1 day       | -            |

|                                                                          |          |          |
|--------------------------------------------------------------------------|----------|----------|
| - Create entity-relationship diagram                                     | 1 day    | -        |
| - Set up development environments (Frontend and Backend)                 | 1 day    | -        |
| - Frontend and Backend development kickoff                               | -        | 1 day    |
| <b>Sprint 3 - Development Phase</b>                                      |          |          |
| - Develop User Authentication (Frontend and Backend)                     | 2 days   | -        |
| - Implement User Profile Screen (Frontend and Backend)                   | 2 days   | -        |
| - Implement Main App Dashboard/Home Screen (Frontend and Backend)        | -        | 4 days   |
| <b>Sprint 4 - User Profile Management</b>                                |          |          |
| - Implement User Profile Display and Editing (Frontend and Backend)      | 2 days   | -        |
| - Implement Logout Functionality (Frontend and Backend)                  | -        | 2 days   |
| - Implement Profile Creation/Editing Screen (Frontend and Backend)       | -        | 2 days   |
| <b>Sprint 5 - Water Intake Reminder and Fitness Goals Tracking</b>       |          |          |
| - Implement Water Intake Reminder Settings Screen (Frontend and Backend) | 2 days   | -        |
| - Design Fitness Tracking Dashboard (UI)                                 | 1 day    | -        |
| - Implement Water Intake Reminder Settings (Frontend and Backend)        | -        | 3 days   |
| <b>Sprint 6 – Testing, Bug Fixes and Refinement</b>                      |          |          |
| - Collaborate on functionality tests, bug identification, and fixes      | 3.5 days | 3.5 days |
|                                                                          |          |          |
| - Review and optimize code                                               | 1.5 days | -        |
| - Enhance UI/UX                                                          | -        | 1 day    |
| - Conduct performance testing                                            | 1 day    | 1 day    |

# DESIGN AND ARCHTECTURE

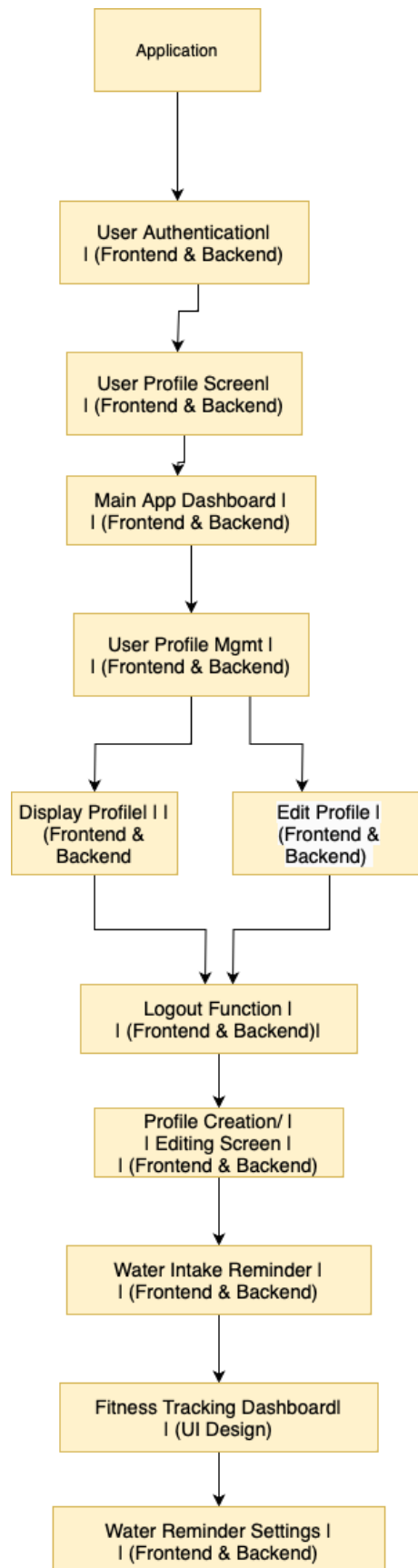
## ER Diagram



## UML

In this UML Activity Diagram:

- Each box represents a specific activity or task.
- Arrows represent the flow of control from one activity to another.
- Activities under "User Profile Mgmt" are grouped to show related tasks.
- Some activities are split into frontend and backend components to highlight the distinction.
- The Fitness Tracking Dashboard task includes UI design, indicating a separate UI design phase.



# IMPLEMENTATION

## Development Environment and Tools Used:

- **React Native:** React Native served as the primary framework for cross-platform mobile app development, facilitating a single codebase for iOS and Android.
- **Node.js:** Node.js was utilized as the backend server, providing a scalable runtime environment for server-side scripting.
- **Redux:** Redux was employed for state management within the React Native application, ensuring a predictable and centralized state.
- **MySQL:** MySQL was chosen as the relational database management system for storing and retrieving user-related data.
- **Jest:** Jest was the selected testing framework for conducting unit and integration testing, ensuring the reliability and stability of the application.
- **Mocha:** Mocha was utilized as the automation testing framework for the application.

## Major Components/Modules:

- **User Authentication Module:** Responsible for secure user registration, login, and authentication processes.
- **Profile Management Module:** Manages user profiles, enabling the input and modification of personal information, preferences, and health-related data.
- **Water Intake Reminder Module:** Implements the feature for customizable water intake reminders to promote daily hydration.
- **Fitness Goal Tracking Module:** Developed a comprehensive system for setting, tracking, and achieving various fitness goals.
- **Fitness Video Access Module:** Introduces a new module enabling users to access a library of fitness videos for guided workout routines and wellness practices.

## Challenges Faced During Implementation:

- **Cross-Platform Compatibility:** Ensuring a consistent user experience across iOS and Android platforms demanded meticulous attention to platform-specific nuances.
- **Real-Time Updates:** Implementing real-time updates for features like progress tracking and notifications required synchronization between the frontend and backend.
- **Scalability:** Scaling the application to accommodate a potentially large user base presented challenges that required strategic planning for optimal performance.
- **Video Integration:** Integrating fitness videos posed challenges related to streaming, playback, and overall performance optimization.

# TESTING

## Manual Testing

### Sprint 1: UI Design and Wireframing

| Test Case ID | Test Case Description                                                 | Preconditions | Test Steps                                                                           | Expected Result                                                     |
|--------------|-----------------------------------------------------------------------|---------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 1.1          | Ensure all stakeholders' inputs are collected.                        | - N/A         | 1. Gather input from stakeholders.                                                   | Stakeholders' inputs are documented.                                |
| 1.2          | Verify that non-functional requirements are considered.               | - N/A         | 1. Review non-functional requirements (e.g., responsiveness, accessibility).         | Non-functional requirements are incorporated into design decisions. |
| 1.3          | Check for error messages for missing or unclear UI requirements.      | - N/A         | 1. Review UI requirements documentation.                                             | Clear documentation with no missing elements.                       |
| 2.1          | Confirm wireframes cover all UI elements in requirements.             | - N/A         | 1. Review wireframes against UI requirements.                                        | Wireframes align with UI requirements.                              |
| 2.2          | Validate wireframes' understandability for all team members.          | - N/A         | 1. Gather feedback on wireframes from both technical and non-technical team members. | Wireframes are easily understandable by all team members.           |
| 2.3          | Ensure error messages for missing or incomplete wireframes.           | - N/A         | 1. Validate completeness of wireframes.                                              | No missing or incomplete wireframes.                                |
| 3.1          | Check for error messages or warnings in the design iteration process. | - N/A         | 1. Iteratively review and refine design.                                             | No errors or warnings are present in the design iterations.         |
| 3.2          | Ensure stakeholders are informed of errors or inconsistencies.        | - N/A         | 1. Communicate design errors or inconsistencies to stakeholders.                     | Stakeholders are aware and informed of any design issues            |



## Sprint 2: Database Schema Design and Frontend/Backend Kickoff

| Test Case ID | Test Case Description                                               | Preconditions | Test Steps                                                          | Expected Result                                                   |
|--------------|---------------------------------------------------------------------|---------------|---------------------------------------------------------------------|-------------------------------------------------------------------|
| 4.1          | Verify database schema alignment with application data model.       | - N/A         | 1. Review the database schema against the application's data model. | Database schema aligns with the data model.                       |
| 4.2          | Check for appropriate error messages during database setup.         | - N/A         | 1. Simulate errors during database setup.                           | Appropriate error messages are displayed for each scenario.       |
| 4.3          | Ensure error handling for invalid database structure definitions.   | - N/A         | 1. Intentionally provide invalid structure definitions.             | System gracefully handles invalid structure definitions.          |
| 5.1          | Ensure error messages for incorrect relationships or cardinalities. | - N/A         | 1. Intentionally set incorrect relationships.                       | Error messages indicate incorrect relationships.                  |
| 5.2          | Verify system alerts for potential integrity violations.            | - N/A         | 1. Intentionally violate data integrity.                            | System alerts users to potential integrity violations.            |
| 5.3          | Check for errors in depicting relationships and cardinalities.      | - N/A         | 1. Review graphical depiction of relationships.                     | Relationships and cardinalities are accurately depicted.          |
| 6.1          | Check for errors when dependencies cannot be installed.             | - N/A         | 1. Intentionally fail dependency installation.                      | Appropriate error messages are displayed for failed dependencies. |
| 6.2          | Verify setup process gracefully handles configuration errors.       | - N/A         | 1. Intentionally provide incorrect configuration.                   | Setup process gracefully handles configuration errors.            |
| 6.3          | Ensure error messages for failures in                               | - N/A         | 1. Simulate environment setup failures.                             | Appropriate error messages for failed                             |

|     |                                                              |       |                                                   |                                                           |
|-----|--------------------------------------------------------------|-------|---------------------------------------------------|-----------------------------------------------------------|
|     | environment setup.                                           |       |                                                   | environment setup.                                        |
| 7.1 | Ensure appropriate error responses for invalid API requests. | - N/A | 1. Make invalid API requests.                     | Appropriate error responses for invalid requests.         |
| 7.2 | Verify frontend gracefully handles errors in data fetching.  | - N/A | 1. Simulate errors during frontend data fetching. | Frontend gracefully handles errors and provides feedback. |
| 7.3 | Confirm error responses for failed endpoint access.          | - N/A | 1. Attempt to access non-existing endpoints.      | Appropriate error responses for failed endpoint access    |

### Sprint 3: Development Phase

| Test Case ID | Test Case Description                                                            | Preconditions                  | Test Steps                                      | Expected Result                                                     |
|--------------|----------------------------------------------------------------------------------|--------------------------------|-------------------------------------------------|---------------------------------------------------------------------|
| 8.1          | Check for error messages when user registration fails.                           | - N/A                          | 1. Attempt user registration with invalid data. | Appropriate error messages are displayed for registration failures. |
| 8.2          | Verify appropriate error responses for failed login attempts.                    | - N/A                          | 1. Attempt login with incorrect credentials.    | Appropriate error responses for failed login attempts.              |
| 8.3          | Ensure error handling for authentication token generation issues.                | - N/A                          | 1. Simulate issues during token generation.     | System handles token generation issues gracefully.                  |
| 9.1          | Ensure error handling for scenarios where user profile data cannot be retrieved. | User has a registered account. | 1. Invalidate user profile data retrieval.      | Appropriate error responses for failed profile data retrieval.      |
| 9.2          | Verify errors during profile updates are                                         | User is logged in.             | 1. Intentionally fail profile updates.          | User is informed of errors during profile updates.                  |

|      |                                                               |                                |                                            |                                                                |
|------|---------------------------------------------------------------|--------------------------------|--------------------------------------------|----------------------------------------------------------------|
|      | communicated to the user.                                     |                                |                                            |                                                                |
| 9.3  | Confirm error responses for failed profile data retrieval.    | User has a registered account. | 1. Invalidate user profile data retrieval. | Appropriate error responses for failed profile data retrieval. |
| 10.1 | Check for error messages when the dashboard cannot be loaded. | User is logged in.             | 1. Intentionally fail dashboard loading.   | Appropriate error messages for failed dashboard loading.       |
| 10.2 | Verify error responses provided for failed data fetching.     | User is logged in.             | 1. Intentionally fail data fetching.       | Appropriate error responses for failed data fetching.          |
| 10.3 | Ensure error handling for dashboard rendering issues.         | User is logged in.             | 1. Simulate dashboard rendering issues.    | System gracefully handles dashboard rendering issues           |

#### Sprint 4: User Profile Management

| Test Case ID | Test Case Description                                                        | Preconditions      | Test Steps                             | Expected Result                                         |
|--------------|------------------------------------------------------------------------------|--------------------|----------------------------------------|---------------------------------------------------------|
| 11.1         | Ensure error handling for cases where user profiles cannot be edited.        | User is logged in. | 1. Invalidate user profile editing.    | Appropriate error responses for failed profile editing. |
| 11.2         | Verify errors during profile editing are communicated to the user.           | User is logged in. | 1. Intentionally fail profile editing. | User is informed of errors during profile editing.      |
| 11.3         | Confirm error responses for failed profile editing.                          | User is logged in. | 1. Invalidate user profile editing.    | Appropriate error responses for failed profile editing. |
| 12.1         | Check for errors during the logout process and display appropriate messages. | User is logged in. | 1. Intentionally fail logout process.  | Appropriate error messages for logout failure.          |

|      |                                                                                 |                    |                                                   |                                                          |
|------|---------------------------------------------------------------------------------|--------------------|---------------------------------------------------|----------------------------------------------------------|
| 12.2 | Verify attempting to log out without proper authentication results in an error. | - N/A              | 1. Attempt to log out without authentication.     | Appropriate error messages for unauthorized logout.      |
| 12.3 | Ensure error handling for logout failure scenarios.                             | User is logged in. | 1. Simulate logout failure scenarios.             | Appropriate error responses for logout failure.          |
| 13.1 | Ensure error messages displayed for invalid profile creation attempts.          | - N/A              | 1. Attempt to create a profile with invalid data. | Appropriate error messages for invalid profile creation. |
| 13.2 | Verify appropriate error responses for failed profile editing.                  | User is logged in. | 1. Intentionally fail profile editing.            | Appropriate error responses for failed profile editing.  |
| 13.3 | Confirm error messages for profile creation failures.                           | - N/A              | 1. Simulate profile creation failure scenarios.   | Appropriate error messages for profile creation failures |

### Sprint 5: Water Intake Reminder and Fitness Goals Tracking

| Test Case ID | Test Case Description                                                       | Preconditions      | Test Steps                                            | Expected Result                                                  |
|--------------|-----------------------------------------------------------------------------|--------------------|-------------------------------------------------------|------------------------------------------------------------------|
| 14.1         | Check for error messages when saving water intake reminder settings fails.  | User is logged in. | 1. Intentionally fail saving reminder settings.       | Appropriate error messages for failed reminder setting updates.  |
| 14.2         | Verify errors during reminder setting updates are communicated to the user. | User is logged in. | 1. Simulate errors during reminder setting updates.   | User is informed of errors during reminder setting updates.      |
| 14.3         | Ensure error handling for failed reminder setting updates.                  | User is logged in. | 1. Intentionally fail saving reminder settings.       | Appropriate error responses for failed reminder setting updates. |
| 15.1         | Ensure error handling for scenarios where the fitness                       | User is logged in. | 1. Simulate failure in loading the fitness dashboard. | Appropriate error messages for failed dashboard rendering.       |

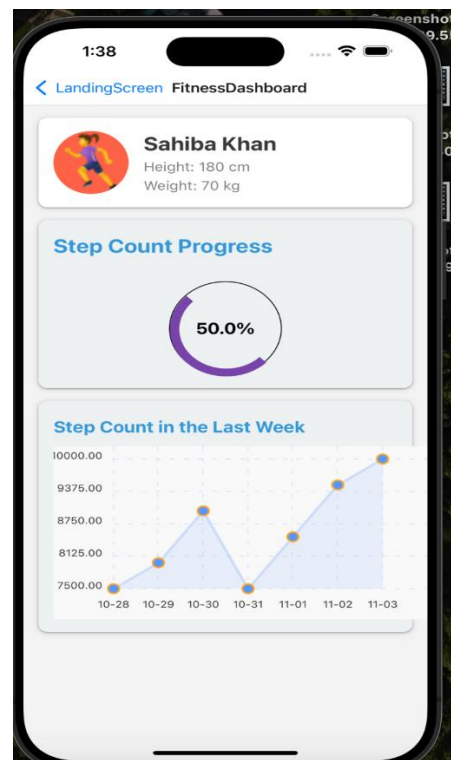
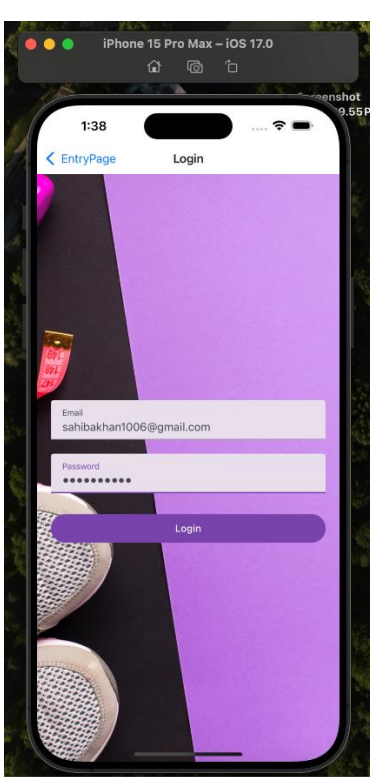
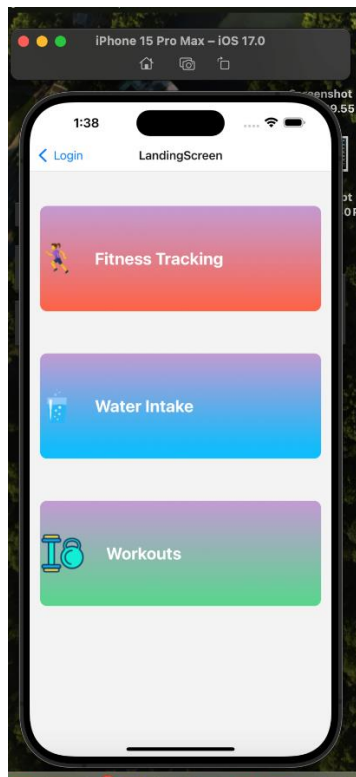
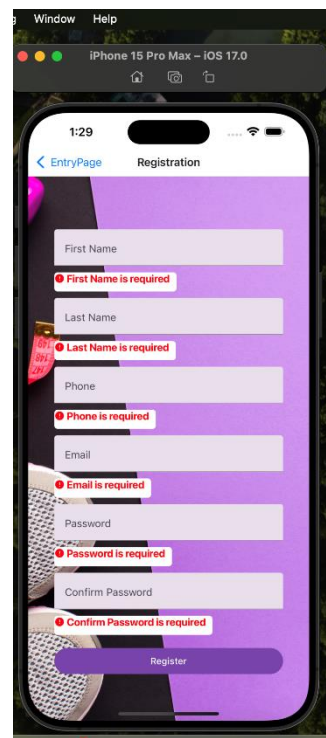
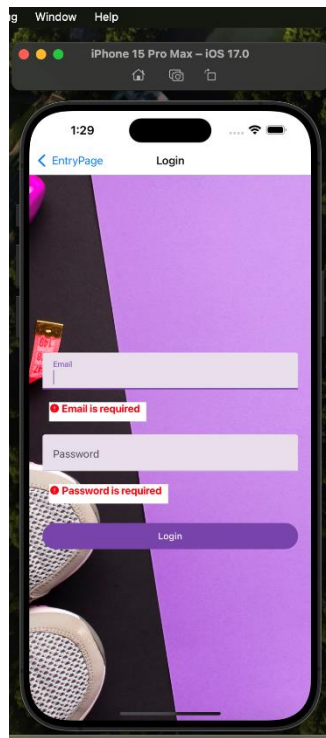
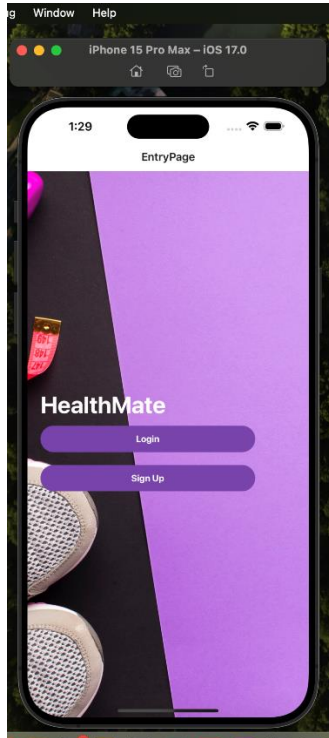
|      |                                                                                   |                    |                                                            |                                                                 |
|------|-----------------------------------------------------------------------------------|--------------------|------------------------------------------------------------|-----------------------------------------------------------------|
|      | dashboard cannot be displayed.                                                    |                    |                                                            |                                                                 |
| 15.2 | Verify that errors are communicated during UI interactions with fitness tracking. | User is logged in. | 1. Intentionally fail UI interactions in fitness tracking. | User is informed of errors during UI interactions.              |
| 15.3 | Confirm error messages for failed dashboard rendering.                            | User is logged in. | 1. Simulate failure in loading the fitness dashboard.      | Appropriate error messages for failed dashboard rendering.      |
| 16.1 | Check for error messages when reminders fail to trigger.                          | User is logged in. | 1. Intentionally fail reminder triggering.                 | Appropriate error messages for failed reminder triggering.      |
| 16.2 | Verify errors during reminder processing are logged for investigation.            | User is logged in. | 1. Monitor system logs during reminder processing.         | Errors during reminder processing are logged for investigation. |
| 16.3 | Ensure error handling for reminder setting failures.                              | User is logged in. | 1. Intentionally fail reminder setting.                    | Appropriate error responses for failed reminder setting         |

### Sprint 6: Testing and Bug Fixes

| Test Case ID | Test Case Description                                                                | Preconditions | Test Steps                                                   | Expected Result                                            |
|--------------|--------------------------------------------------------------------------------------|---------------|--------------------------------------------------------------|------------------------------------------------------------|
| 17.1         | Verify that appropriate error messages are displayed for failed functionality tests. | - N/A         | 1. Execute functionality tests with intentional failures.    | Appropriate error messages for failed functionality tests. |
| 17.2         | Check for errors when validating data integrity and communicate issues clearly.      | - N/A         | 1. Perform data integrity tests with intentional violations. | Clear communication of data integrity issues.              |
| 17.3         | Confirm error messages for failed end-to-end tests.                                  | - N/A         | 1. Execute end-to-end tests with intentional failures.       | Appropriate error messages for failed end-to-end tests.    |

|      |                                                                              |       |                                                         |                                                |
|------|------------------------------------------------------------------------------|-------|---------------------------------------------------------|------------------------------------------------|
| 18.1 | Ensure that errors identified during testing are logged for developers.      | - N/A | 1. Log errors identified during testing.                | Errors are logged for developer investigation. |
| 18.2 | Verify that errors mentioned in bug reports are resolved in a timely manner. | - N/A | 1. Track and resolve reported bugs.                     | Reported bugs are resolved in a timely manner. |
| 18.3 | Confirm error messages for identified bugs.                                  | - N/A | 1. Verify error messages associated with resolved bugs. | Error messages for resolved bugs are confirmed |

## SCREENSHOTS



MySQL Workbench

Local instance 3306 - Warning - not supported

Administration Schemas SQL File 1\*

SCHEMAS

Filter objects

- healthmate
- user\_details
- users
- water\_reminder

Object Info Session

Table: **user\_details**

Columns:

- id: int AI PK
- user\_id: int
- height: varchar(10)
- weight: int
- step\_count: int
- water\_reminder: tinyint(1)

SQL Editor

```

1 • USE healthMate;
2
3 • Select * from user_details;
4
5

```

Result Grid

| id   | user_id | height | weight | step_count | water_remind... |
|------|---------|--------|--------|------------|-----------------|
| 1    | 1       | NULL   | NULL   | NULL       | NULL            |
| 2    | 2       | 180    | 70     | 5000       | 1               |
| NULL | NULL    | NULL   | NULL   | NULL       | NULL            |

user\_details 1

Apply Revert

Action Output

|   | Time     | Action                                   | Response          | Duration / Fetch Time   |
|---|----------|------------------------------------------|-------------------|-------------------------|
| 2 | 13:39:28 | Select * from user_details LIMIT 0, 1000 | 2 row(s) returned | 0.011 sec / 0.000005... |

Query Completed

MySQL Workbench

Local instance 3306 - Warning - not supported

Administration Schemas SQL File 1\*

SCHEMAS

Filter objects

- healthmate
  - Tables
    - Activity
    - user\_details

Object Info Session

Table: **Activity**

Columns:

- activity\_id: int AI PK
- user\_id: int
- steps\_taken: int
- activity\_date: date

SQL Editor

```

1 • USE healthMate;
2
3 • Select * from activity;
4
5

```

Result Grid

| activity_id | user_id | steps_taken | activity_date |
|-------------|---------|-------------|---------------|
| 1           | 2       | 7500        | 2023-10-28    |
| 2           | 2       | 8000        | 2023-10-29    |
| 3           | 2       | 9000        | 2023-10-30    |
| 4           | 2       | 7500        | 2023-10-31    |
| 5           | 2       | 8500        | 2023-11-01    |
| 6           | 2       | 9500        | 2023-11-02    |
| 7           | 2       | 10000       | 2023-11-03    |
| NULL        | NULL    | NULL        | NULL          |

activity 2

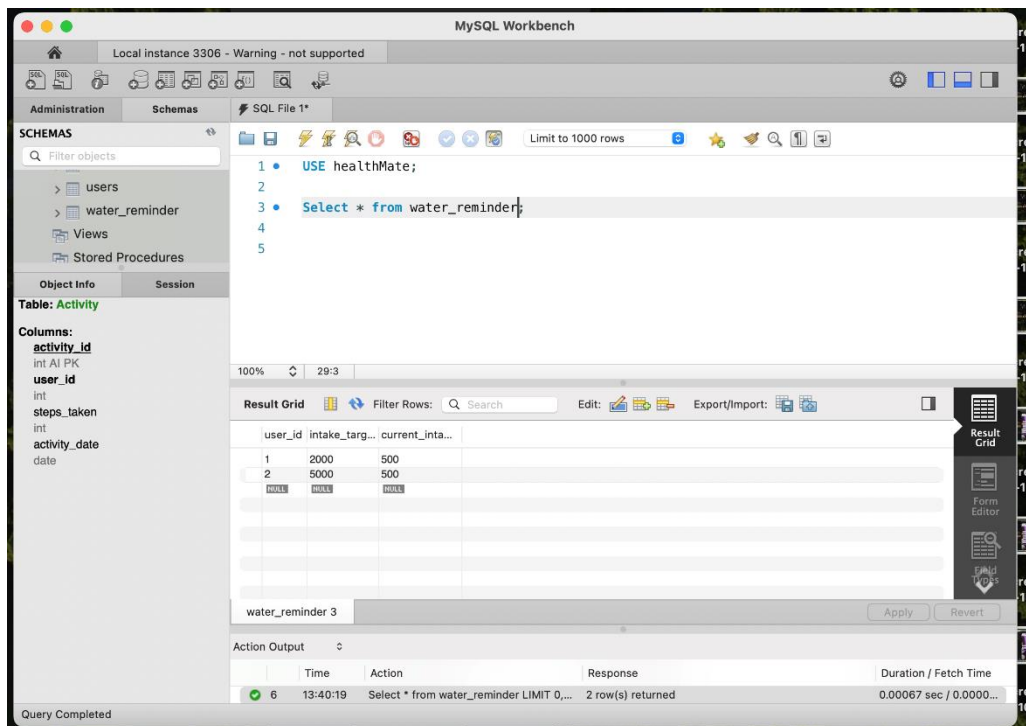
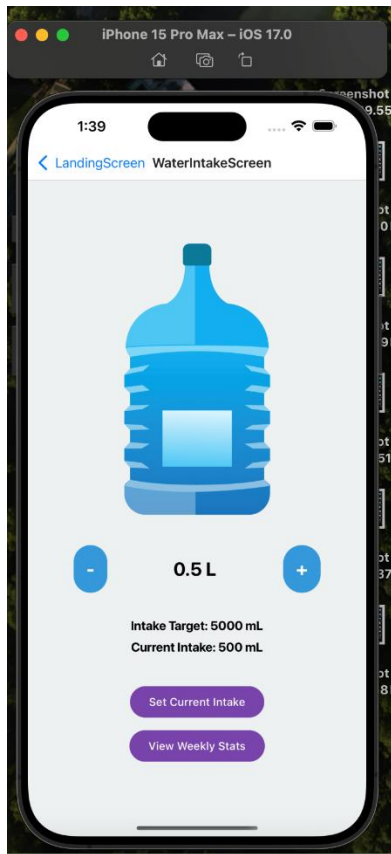
Apply Revert

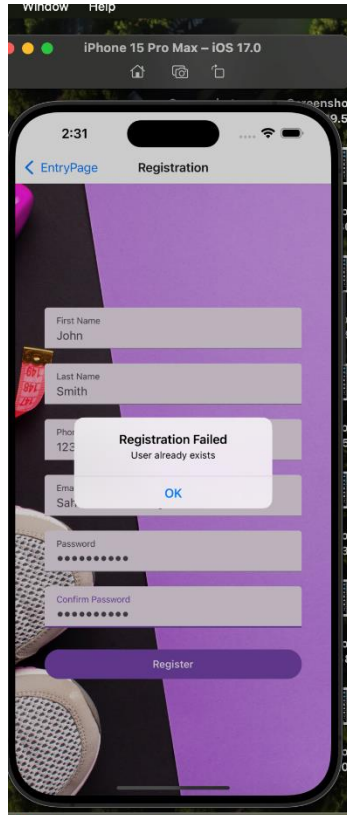
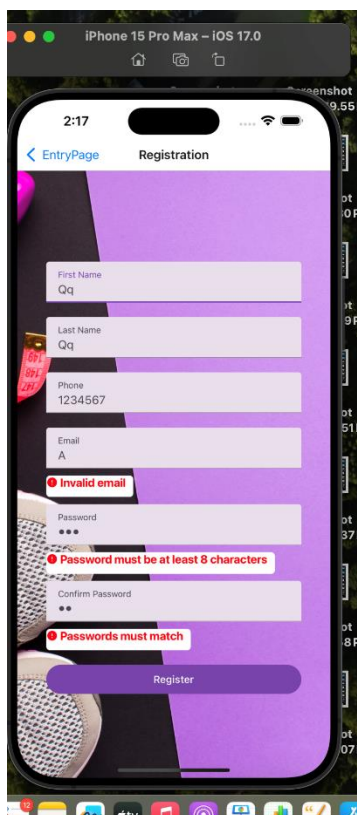
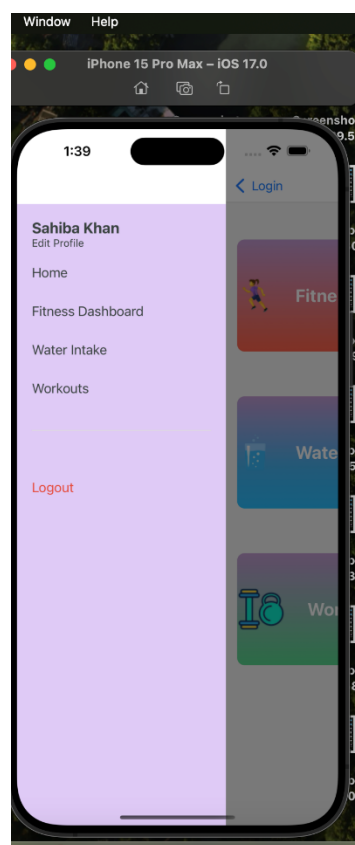
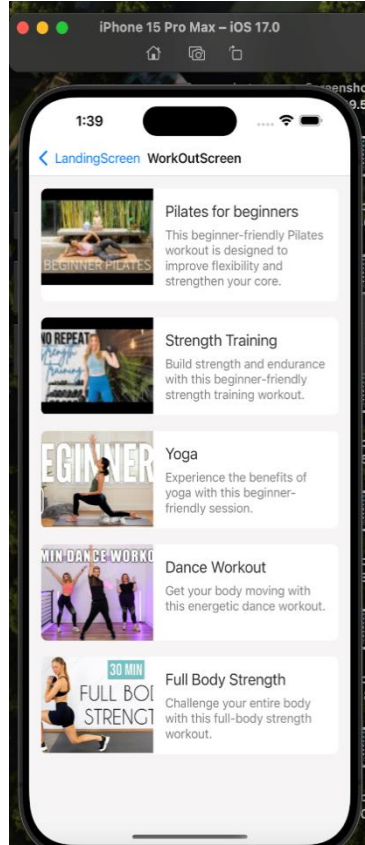
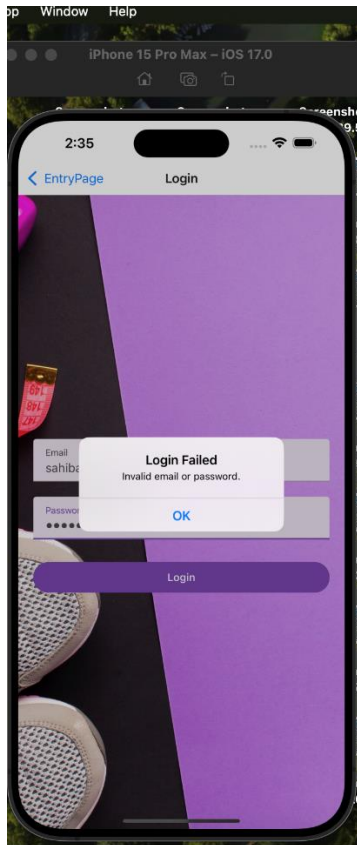
Action Output

|   | Time     | Action                               | Response          | Duration / Fetch Time   |
|---|----------|--------------------------------------|-------------------|-------------------------|
| 4 | 13:39:54 | Select * from activity LIMIT 0, 1000 | 7 row(s) returned | 0.00070 sec / 0.0000... |

Query Completed







## Automation Testing

### Test Cases:

- **Login with Valid Credentials:**
  - **Test Purpose:** Validate that users can successfully authenticate using correct credentials.
  - **Expected Outcome:** The login request should return a status of 200, indicating success. The response should include a message confirming successful login, along with an authentication token and the user's ID.
- **Login with Invalid Credentials:**
  - **Test Purpose:** Ensure that the system rejects login attempts with incorrect credentials.
  - **Expected Outcome:** The login request should return a status of 401, indicating unauthorized access. The response should include an error message indicating invalid credentials.
- **Register a New User with Valid Data:**
  - **Test Purpose:** Verify that new users can be registered successfully with valid information.
  - **Expected Outcome:** The registration request should return a status of 200, indicating success. The response should include a message confirming successful registration.
- **Attempt to Register a User with an Existing Email:**
  - **Test Purpose:** Confirm that the system prevents registration attempts with email addresses already associated with existing users.
  - **Expected Outcome:** The registration request should return a status of 409, indicating a conflict due to the existing user. The response should include an error message indicating that the user already exists.
- **Insert User Details:**
  - **Test Purpose:** Validate that user-specific details, such as height, weight, steps, and water reminder, can be successfully inserted.
  - **Expected Outcome:** The request to insert user details should return a status of 200, indicating success. The response should include a message confirming successful data insertion.

### Frameworks Used:

- **Mocha:**
  - **Description:** A flexible JavaScript testing framework for organizing test suites and cases.
  - **Advantages:** Simplicity, flexibility, and wide adoption in the JavaScript ecosystem.

- **Chai:**
  - **Description:** An assertion library providing a fluent interface for clear and readable test assertions.
  - **Advantages:** Expressive syntax, multiple assertion styles, and widespread usage.
- **Supertest:**
  - **Description:** A library for testing HTTP assertions, commonly used with Mocha and Chai for API testing.
  - **Advantages:** Simplifies sending HTTP requests, integrates well with Mocha and Chai.

The screenshot shows a VS Code editor with a project named 'healthMate-BE-main'. The Explorer sidebar on the left shows the file structure: 'HEALTHMATE-BE-MAIN' containing 'node\_modules', '.gitattributes', 'app.js', 'package-lock.json', 'package.json', and 'test.js'. The main editor area shows the 'test.js' file with the following code:

```

38 // ...
39 res.status(400).json({ error: 'user_id parameter is required' });
40 return;
41 }
42 // Modify the SQL query to retrieve user details for the specified user_id
43 const sql = `SELECT * FROM users WHERE user_id = ?`;
44
45 // Execute the SQL query with the user_id parameter
46 connection.query(sql, [user_id], (err, results) => {
47   if (err) {
48     console.error('Error querying data:', err);
49     res.status(500).json({ error: 'Internal server error' });
50   }
51 });

```

Below the code editor, the TERMINAL panel shows the output of the test suite:

```

Server is running on port 3000

Connected to MySQL database
HealthMate API Tests
  ✓ should login with valid credentials
  ✓ should not login with invalid credentials
  ✓ should register a new user with valid data
  ✓ should not register a user with an existing email
  ✓ should insert user details

5 passing (95ms)

```

The status bar at the bottom indicates the current file is 'Ln 3, Col 22', using 'Spaces: 2', 'UTF-8' encoding, and 'LF' line endings. It also shows the active language is 'Babel JavaScript' and the 'Go Live' button is available.

# MAINTAINANCE

## 1. Updates, Patches, and Bug Fixes:

- **Release Schedule:** Bug fixes will be addressed promptly and released as part of scheduled maintenance updates or hotfix releases.
- **Bug Tracking and Resolution:** An issue tracking system, such as Jira, will be used to log and prioritize reported bugs and issues. Bugs will be categorized based on severity and impact on user experience. Critical bugs affecting functionality or security will be addressed immediately, while less critical issues will follow the regular release cycle.
- **User Feedback and Beta Testing:** User feedback channels will be actively monitored, and insights from users will be considered for continuous improvement. Beta testing programs will be conducted for major updates to identify potential issues and gather user feedback before a widespread release.

## 2. Scalability and Futureproofing:

- **Scalability Considerations:** Continuous monitoring of user engagement and server performance to anticipate and address scalability challenges. Periodic load testing to assess the application's capacity and identify potential bottlenecks. Infrastructure components will be designed with scalability in mind, allowing for seamless scaling as user demand increases.
- **Technology Stack Updates:** Regular evaluations of the technology stack to identify opportunities for updating to newer versions or integrating emerging technologies. Gradual adoption of the latest stable releases of React Native, Node.js, and other relevant frameworks to benefit from performance improvements, security patches, and new features.
- **Security Audits:** Regular security audits and vulnerability assessments to identify and address potential security risks. Immediate application of security patches and updates to safeguard user data and maintain a secure environment.

## 3. Communication and User Education:

- **Transparent Communication:** Clear communication with users about upcoming updates, new features, and bug fixes through release notes, in-app notifications, and official communication channels.
- **User Education:** Continuous education through in-app guides, tutorials, and FAQs to inform users about new features, best practices, and how to make the most of the HealthMate app.

#### **4. Data Backup and Recovery:**

- **Regular Data Backups:**
- Implementation of regular data backup procedures to prevent data loss in the event of unforeseen issues or server failures.
- **Disaster Recovery Plan:**
- Development and maintenance of a comprehensive disaster recovery plan to ensure a rapid response to critical incidents and minimize downtime.

#### **5. User Support and Community Engagement:**

- **User Support Channels:**
- A dedicated user support team to address user queries, troubleshoot issues, and aid.
- Community forums and social media engagement to foster a sense of community and encourage user collaboration.

The ongoing maintenance plan for HealthMate emphasizes a proactive approach to updates, patches, and bug fixes. Scalability considerations, technology stack updates, and security audits are integrated into the plan to ensure the application remains robust, secure, and aligned with evolving user needs. Transparent communication, user education, data backup procedures, and strong user support contribute to the overall reliability and success of the HealthMate mobile application.

# CONCLUSION

## Outcome

The HealthMate project has successfully delivered a feature-rich mobile application aimed at assisting users in managing their fitness and health goals. The app encompasses user-friendly interfaces, robust authentication, water intake reminders, fitness goal tracking, and a diverse library of fitness videos. The project adhered to a well-structured timeline, meeting its objectives of designing, developing, testing, and deploying the HealthMate app within the planned duration.

## Achievements

- **User-Centric Design:** The app achieved a user-centric design, providing a seamless experience for user registration, profile management, and access to various health and fitness features.
- **Comprehensive Features:** HealthMate offers a comprehensive set of features, including water intake reminders, fitness goal tracking, and a fitness video library, addressing different aspects of users' health and wellness.
- **Cross-Platform Compatibility:** Utilizing React Native allowed for the creation of a cross-platform application, ensuring a consistent experience for both Android and iOS users.
- **Technical Feasibility:** The chosen technology stack, including React Native, Node.js, Redux, MySQL, and Jest, contributed to the technical feasibility of the project, providing a scalable, efficient, and well-tested solution.
- **Positive Feasibility Study:** The feasibility study indicated positive results, highlighting technical, operational, and economic feasibility. The technology choices, revenue streams, and market positioning align with the expected positive return on investment.

## Lessons Learned:

- **Continuous User Feedback:** Establishing ongoing channels for user feedback and engagement is crucial for refining features and addressing user needs effectively.
- **Iterative Development:** Adopting an iterative development approach allows for flexibility in responding to changing requirements and user preferences throughout the project lifecycle.
- **Scalability Considerations:** Scalability planning should be integral to the project from the outset to accommodate potential increases in user demand.

## Areas for Improvement:

- **Enhanced Testing Coverage:** While testing coverage was substantial, improvements can be made by expanding test scenarios, especially in edge cases and user-specific scenarios.

- **Streamlined Deployment:** Investigate ways to optimize the deployment process further, ensuring quicker and more efficient releases while minimizing downtime.
- **User Education:** Enhance in-app educational resources to guide users on maximizing the utilization of features, promoting a better understanding of the app's capabilities.

The HealthMate project has achieved its primary objectives, providing a valuable tool for users to manage their health and fitness goals. The positive feasibility study and user feedback highlight the project's success. Continuous improvement, iterative development, and a focus on scalability will be integral to maintaining and enhancing the app's success in the dynamic health and fitness market.



## RECOMMENDATIONS

- **Integration of Wearable Devices:** Explore the integration of wearable devices to enhance data accuracy and provide users with real-time health metrics. This could include syncing with popular fitness trackers and smartwatches to capture additional health data such as heart rate, sleep patterns, and activity levels.
- **Social Engagement Features:** Introduce social engagement features to foster a sense of community among HealthMate users. This could include features like user forums, challenges, and the ability to connect with friends for shared fitness goals, creating a more interactive and supportive user environment.
- **Advanced Analytics and Insights:** Develop advanced analytics tools to provide users with detailed insights into their health and fitness progress. Incorporate visualizations, trend analysis, and personalized recommendations based on users' historical data to offer a more comprehensive understanding of their wellness journey.
- **Enhanced Personalization:** Implement machine learning algorithms to enhance personalization by tailoring content recommendations, fitness plans, and reminders based on individual user preferences, behavior, and health data. This will contribute to a more personalized and adaptive user experience.
- **Incorporate Telehealth Features:** Explore the integration of telehealth features, allowing users to connect with health professionals for virtual consultations, personalized advice, and remote monitoring. This addition can contribute to a holistic approach to health management within the HealthMate ecosystem.
- **Gamification Elements:** Introduce gamification elements to make the fitness journey more engaging. Features such as achievement badges, rewards, and leaderboards can motivate users to stay consistent with their health and fitness routines, turning the app into a more interactive and enjoyable experience.
- **Localization and Global Expansion:** Consider localization efforts to make the app accessible to a global audience by supporting multiple languages and cultural preferences. This will expand the app's reach and cater to users from diverse backgrounds.
- **Continuous User Feedback Mechanism:** Establish a systematic and continuous user feedback mechanism within the app to gather insights, preferences, and suggestions directly from users. This feedback loop will be invaluable for understanding evolving user needs and shaping future enhancements.
- **Augmented Reality (AR) Integration:** Explore the integration of augmented reality features for immersive fitness experiences. AR can be utilized for interactive workout routines, guided exercises, and enhancing the overall user engagement within the app.
- **Enhanced Security Measures:** Implement additional security measures, such as two-factor authentication, to further enhance the protection of user accounts and sensitive health data. Regular security audits should be conducted to identify and address potential vulnerabilities.
- **Continuous Performance Optimization:** Continuously optimize the app's performance to ensure responsiveness, quick load times, and smooth user interactions. Regularly

review and update the technology stack to leverage the latest advancements and improvements in mobile development.

These recommendations are designed to propel HealthMate to the forefront of health and fitness applications, ensuring it remains innovative, user-centric, and aligned with emerging trends in the health and wellness industry.

## REFERENCES

### Frameworks and libraries used in the project:

<https://reactnative.dev/>

<https://nodejs.org/en>

<https://redux.js.org/>

<https://www.mysql.com/>

<https://jestjs.io/>

<https://mochajs.org/>

<https://healthmate123.atlassian.net/jira/software/projects/HEAL/boards/1/backlog?selectedIssue=HEAL-5>

### Other Sources:

<https://www.clearpointstrategy.com/blog/18-key-performance-indicators>

<https://www.investopedia.com/terms/k/kpi.asp>

<https://confluence.atlassian.com/jira>

<https://perpet.io/blog/what-should-be-ui-ux-design-of-fitness-app/>

# APPENDICES

The screenshot shows the VS Code editor with the `healthMate` project open. The Explorer panel on the left shows the project structure, with `FitnessDashboard.js` selected under the `components` folder. The main editor displays the code for `FitnessDashboard.js`, which includes imports for React, navigation, and data fetching. A comment at the top reads: `healthMate > src > components > FitnessDashboard.js > useEffect() callback > fetchStepCountData > response`. The code defines a `FitnessDashboard` component that uses `useState` and `useEffect` to manage state and fetch data. The `useEffect` hook calls `fetchStepCountData` and sets the `stepCount` state. The `handleLogout` function calls `clearUserSession` and resets the navigation. The `useEffect` hook also calls `fetchStepCountData` and sets the `stepCount` state. The `handleLogout` function calls `clearUserSession` and resets the navigation. The `useEffect` hook also calls `fetchStepCountData` and sets the `stepCount` state.

The screenshot shows the VS Code editor with the `healthMate` project open. The Explorer panel on the left shows the project structure, with `WaterIntakeScreen.js` selected under the `components` folder. The main editor displays the code for `WaterIntakeScreen.js`, which includes imports for React, navigation, and data fetching. A comment at the top reads: `healthMate > src > components > WaterIntakeScreen.js > WaterIntakeScreen > setWaterIntake > response`. The code defines a `WaterIntakeScreen` component that uses `useState` and `useEffect` to manage state and fetch data. The `useEffect` hook calls `loadFonts` and sets the `fontLoaded` state. The `increaseWater` function calls `setWaterQuantity` and sets the `waterQuantity` state. The `decreaseWater` function calls `setWaterQuantity` and sets the `waterQuantity` state. The `useEffect` hook also calls `loadFonts` and sets the `fontLoaded` state.

```

287 res.status(404).json({ error: 'User not found' });
288 } else {
289   // Send the query results as a JSON response
290   res.json(results[0]); // Assuming you expect only one user to match the user_id
291 }
292 });
293
294 // Define an API endpoint to store step count data
295 app.post('/api/stepcount', (req, res) => {
296   const { user_id, date, steps } = req.body;
297
298   if (!user_id || !date || steps === undefined) {
299     res.status(400).json({ error: 'user_id, date, and steps parameters are required' });
300     return;
301   }
302
303   // Modify the SQL query to insert step count data into the database
304   const sql = `INSERT INTO Activity (user_id, date, steps) VALUES (?, ?, ?)`;
305
306   // Execute the SQL query with the provided parameters
307   connection.query(sql, [user_id, date, steps], (err, results) => {
308     if (err) {
309       console.error('Error inserting step count data:', err);
310       res.status(500).json({ error: 'Internal server error' });
311     } else {
312       // Insert successful, no need to check results.affectedRows
313       res.json({ message: 'Step count data inserted successfully' });
314     }
315   });
316 }
317
318 // Define an API endpoint to retrieve Activity data by user_id
319 app.get('/api/Activity', (req, res) => {
320   const user_id = req.query.user_id;
321
322   if (!user_id) {
323     res.status(400).json({ error: 'user_id parameter is required' });
324     return;
325   }
326
327   // Modify the SQL query to retrieve Activity data for the specified user_id
328   const sql = `SELECT * FROM Activity WHERE user_id = ?`;

```

## Version control

| Name                  | Last commit message | Last commit date |
|-----------------------|---------------------|------------------|
| ..                    |                     |                  |
| android               | changes             | yesterday        |
| assets                | changes             | 3 days ago       |
| ios                   | changes             | yesterday        |
| rn-appium-test        | changes             | 3 days ago       |
| src                   | commits             | 1 hour ago       |
| .gitignore            | Create .gitignore   | 2 months ago     |
| App.js                | msgs                | 1 hour ago       |
| Font.js               | UI changes          | last month       |
| app.json              | changes             | yesterday        |
| apium-test-run-ios.js | changes             | 3 days ago       |
| babel.config.js       | changes             | 3 days ago       |
| eas.json              | changes             | yesterday        |
| index.js              |                     |                  |
| metro.config.js       |                     |                  |
| package-lock.json     |                     |                  |
| package.json          |                     |                  |

# JIRA board

